JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번: 202413123 **이름:** 조이진 **제출일:** 2026 년 10 월 20 일

서비스 이름: 무비로그 (영화 리뷰 게시판)

기술 조합과 버전: React 19.3.0 + Vite 8.3.4 / FastAPI 0.143.0 + Uvicorn 0.54.0 + PyJWT 2.15.1 / Python 3.12.10, Node.js v24.15.0 / SQLite (Python 표준 sqlite3) / Windows 11 Education

GitHub 저장소 URL: https://github.com/zohijin-k/MovieLog

소스 코드 기준: main / 1af58eb

무비로그는 영화를 본 관람객이 짧은 한 줄 평부터 긴 감상까지 리뷰로 남기고, 다른 사람의 리뷰를 둘러볼 수 있는 게시판입니다. 리뷰는 로그인 없이 누구나 읽을 수 있습니다. 작성은 로그인한 사용자만 할 수 있고, 수정 및 삭제는 작성자 본인만 할 수 있습니다. 이 권한은 서버가 JWT 를 검증하여 실제 작성자인지 확인합니다.

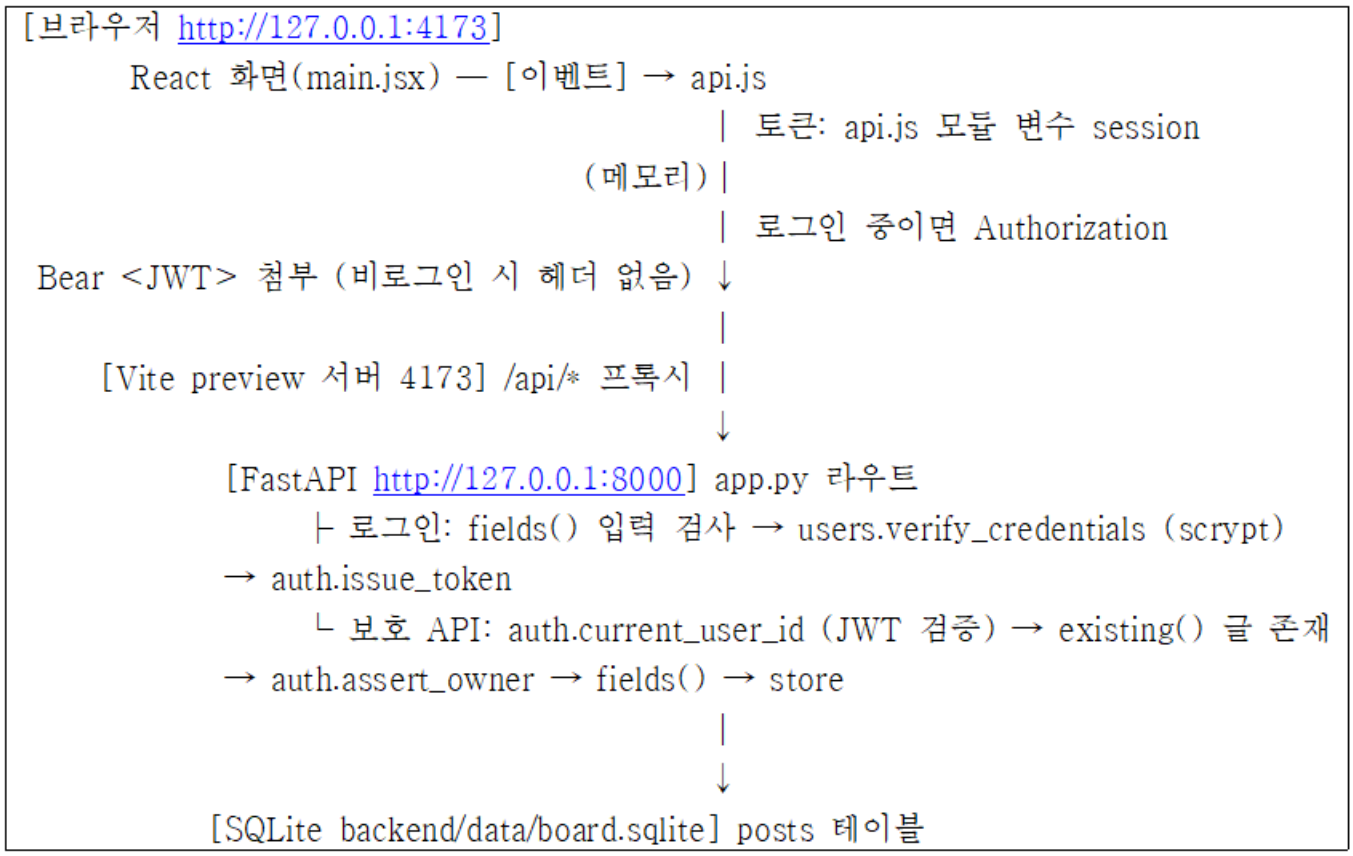
기술 조합은 React 와 FastAPI 를 선택했습니다. FastAPI 는 Python 으로 작성되어 평소 익숙하고, 제공 템플릿이 표준 라이브러리 sqlite3 만으로 DB 를 다루기 때문에 요청이 라우트에서 인증 함수와 저장소까지 흘러가는 과정을 추적하기 쉬웠습니다. 또한 JWT 처리에 PyJWT 를 사용하므로 서명, 만료, 발급자 및 사용 대상 검증을 라이브러리 옵션으로 명확하게 지정할 수 있었습니다. React 는 로그인 사용자, 선택한 글, 입력값을 상태로 관리하고 상태가 바뀌면 화면이 바로 갱신되므로, 로그아웃이나 401 응답 시 화면을 즉시 갱신하기에 적합하다고 판단했습니다.

제공된 템플릿 backend-fastapi 와 frontend-react 를 기반으로 사용했으며, 제출 구조에 맞추기 위해 폴더 이름을 각각 backend, frontend 로 변경했습니다. SQLite 파일 경로는 기본값 backend/data/board.sqlite 를 그대로 사용합니다. 백엔드에서는 auth.py 의 JWT 발급 및 검증과 작성자 권한 검사를 구현하고, app.py 에 기본 422 응답과 예상하지 못한 오류를 공통 JSON 형식으로 바꾸는 예외 처리를 추가했습니다. 프론트엔드에서는 api.js 의 토큰 메모리 저장, Authorization 헤더 첨부, 상태 코드별 처리를 구현하고, main.jsx 의 화면을 영화 리뷰 게시판에 맞게 구성하였습니다. 제공된 게시글 CRUD, SQLite 저장소, 테스트 계정과 비밀번호 검증 함수는 그대로 유지했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/app.py frontend/src/api.js	제공 코드 유지
로그인과 JWT 발급	완료	backend/auth.py frontend/src/api.js	HS256, TTL 기본 90 초
인증과 내 정보 조회	완료	backend/auth.py backend/app.py	-
본인 글 작성·수정·삭제	완료	backend/app.py backend/auth.py frontend/src/api.js	-
만료·로그아웃·오류 표시	완료	frontend/src/api.js frontend/src/main.jsx	로그아웃은 서버에서 토큰을 무효화하지 않음(설계 범위 밖)
DB 데이터 유지	완료	backend/store.py	제공 코드 유지

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.



[로그인 흐름]

1. 로그인 폼이 login(email, password)를 호출하고 POST/api/auth/login 으로 JSON 을 보냅니다.
2. 서버는 계정을 확인한 뒤 JWT 를 발급합니다.
3. 응답의 accessToken 과 user 는 api.js 의 session 변수(메모리)에만 저장됩니다.

[게시글 수정 흐름]

1. 수정 저장 버튼이 api.update 를 호출합니다.
2. request()가 authorizationHeaders()로 Authorization: Bearer<JWT>를 붙여 PUT /api/posts/{id}를 보냅니다.
3. 서버는 토큰 검증 → 글 존재 → 작성자 비교 → 입력 검사 순서로 확인한 뒤 DB 를 갱신합니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유	작성 출처
backend/auth.py _load_secret, _load_ttl (신규)	없음	• .env 의 JWT_SECRET 을 64 자리 hex 에서 32 바이트로 변환 • 빈 값, 예시 키, 형식 오류, 단순 반복 값이면 시작 시 RuntimeError 를 출력 • TOKEN_TTL 도 양의 정수인지 검사	예시, 약한 키로 서버가 실행되는 것을 막기 위함	Claude code
backend/auth.py issue_token	501 NOT_IMPLEMENTED	HS256 으로 sub/iat/exp/iss/aud 를 담아 서명	BE-01	제공 코드 기반 Claude code
backend/auth.py current_user_id, _bearer_token	항상 1 반환	Bearer 추출 → jwt.decode 로 서명/알고리즘/만료/필수 클레임 검증 → sub 형식과 계정 존재 확인	BE-02	제공 코드 기반 Claude code
backend/auth.py assert_owner	검사 없음	로그인 사용자 ID 와 DB authorId 가 다르면 403	BE-03	제공 코드 기반 Claude code
backend/app.py 예외 처리기 3 개 (신규)	ApiError 만 처리	• 422→400 BAD_INPUT • 프레임워크 404/405→JSON • 예상하지 못한 오류 → 500 • INTERNAL_ERROR (스택 비노출)	모든 실패를 {code, message} 형식으로 맞추기 위함	Claude code
backend/app.py login	os.getenv 로 TTL 읽기	검증된 TOKEN_TTL 사용	발급 TTL 과 expiresIn 일치	Claude code
backend/.env.example	설정 예시	키 생성 명령과 "예시 키는 거부됨" 안내 주석 추가	재현성	Claude code
frontend/src/api.js authorizationHeaders, login, canEdit, request, readBody	• TODO • canEdit 항상 true	• 메모리 토큰으로 Bearer 헤더 생성 • 로그인 저장 • 401 시 clearSession • 204 빈 본문은 파싱 생략 • 네트워크 오류 안내	FE-01~03	제공 코드 기반 Claude code
frontend/src/main.jsx describe, run, logout, 화면	• 데모 배너 • 항상 표시되는 로그인 폼/버튼	• 상태 코드/code 별 안내 • 404 시 목록 갱신 • 로그인 사용자 표시 • 토큰 남은 시간 • 비로그인 작성 안내 • 본인 글에만 수정 및 삭제	FE-03~04	제공 코드 기반 Claude code
frontend/src/style.css index.html	기본 스타일	• 오류, 성공 메시지 색 • 사용자 영역 • 제목 변경	주제 반영	제공 코드 기반 Claude code
tools/check_claims.py (신규)	없음	로컬 키로 클레임을 조작한 토큰을 만들어 거부되는지 검사	서명 검사와 별도로 클레임 검사를 확인하기 위해	Claude code

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]

[1] 실행 환경은 Windows 11 Education(10.0.26200), Node.js v24.15.0, npm 11.12.1, Python 3.12.10 입니다. 기술 조합은 React(Vite 8.3.4)와 FastAPI(0.143.0)입니다. git clone 한 뒤, 아래 명령으로 처음부터 설치했습니다.

[2] 백엔드 (PowerShell)

```
cd backend

py -m venv .venv

.venv\Scripts\python.exe -m pip install -r requirements.txt

Copy-Item .env.example .env

.venv\Scripts\python.exe -c "import secrets; print(secrets.token_hex(32))"

(출력된 64자리 값을 .env의 JWT_SECRET에 입력)

.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

[3] 프론트 (PowerShell)

```
cd frontend
npm ci
npm run build
npm run preview
```

[4] 환경설정 값은 JWT_ISSUER=webservice-class, JWT_AUDIENCE=board-api, TOKEN_TTL=90 입니다. JWT_SECRET 은 실행할 때마다 새로 생성하며 저장소와 보고서에는 남기지 않습니다. 예시 키를 그대로 두면 서버가 "JWT_SECRET 이 비어 있거나 예시 값입니다" 오류로 시작을 거부하는 것을 확인했습니다.

[5] npm ci 는 19 개 패키지를 설치했고(취약점 0), npm run build 는 16 개 모듈을 변환해 dist/index.html 과 JS·CSS 를 생성했습니다(종료 코드 0). SQLite 파일은 backend/data/board.sqlite 이며, 서버를 처음 시작할 때 폴더와 테이블이 자동 생성됩니다. 새 폴더에서 빈 목록 [] 조회와 Alice(id 1), Bob(id 2) 로그인 200 을 확인해 DB 와 테스트 계정이 자동으로 준비됨을 확인했습니다(fresh_install_log.txt, build_log.txt).

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / 증거 위치]
npm run preview 로 실행한 http://127.0.0.1:4173 에 접속해 리뷰 목록이 표시되는 것을 확인했습니다. 프론트가 /api 로 보낸 요청은 Vite preview 프록시를 통해 8000 번 백엔드로 전달되었습니다. 4173 을 거친 GET /api/health 는 200 {"status":"ok"}였습니다. 브라우저에서 보낸 GET /api/posts, POST /api/auth/login, GET /api/me 요청과 응답은 browser_T02_T11_T16.txt 에 기록되어 있습니다(그림: T02_screen.png, T10.png).
서버 재시작 후 데이터 유지는 T15 로 확인했습니다. Alice 로 'persist check' 글(15 번)을 작성한 뒤 백엔드를 Ctrl+C 로 종료하고 같은 backend 폴더에서 다시 실행했습니다. GET /api/posts/15 의 제목, 작성자(authorId 1), updatedAt(2026-10-19T06:46:56.606301+00:00)이 재시작 전과 같았습니다(results.md T15 행).

증거 위치: docs/evidence/Test/ 폴더의 fresh_install_log.txt, build_log.txt, browser_T02_T11_T16.txt, results.md, T02_screen.png, T10.png

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3.1 로그인과 JWT 발급

파일 / 함수: backend/auth.py _load_secret(13 행), issue_token(44 행) / backend/app.py login(91 행)

코드 발췌

```python
def issue_token(user):
    # BE-01: 공개 사용자 정보 중 id만 sub로 넣는다. 비밀번호, 해시는 넣지 않는다.
    now = datetime.now(timezone.utc)
    claims = dict(
        sub=str(user["id"]),
        iat=now,
        exp=now + timedelta(seconds=TOKEN_TTL),
        iss=ISSUER,
        aud=AUDIENCE,
    )
    return jwt.encode(claims, SECRET, algorithm=ALGORITHM)
```

동작과 응답 코드 설명

1. 입력: {"email", "password"}
2. 입력 검사: fields()가 문자열, 길이(254/128), 허용 필드를 확인합니다. 실패 시, **400 BAD_INPUT**
3. 계정 확인
   (1) 제공 함수 verify_credentials 가 scrypt 로 비밀번호를 비교합니다.
   (2) 이메일이 없을 때도 같은 연산을 해서 시간 차이를 줄입니다.
   (3) 틀린 비밀번호와 없는 이메일 모두 **401 INVALID_CREDENTIALS** 이며 토큰을 발급하지 않습니다.
4. 발급: 확인된 사용자에게만 issue_token 이 HS256 으로 서명합니다.
   (1) sub: 사용자 ID 문자열("1")
   (2) exp: 현재 + TOKEN_TTL 초 (Unix 초)
   (3) iss: webservice-class
   (4) aud: board-api
5. 토큰에는 비밀번호와 해시가 들어가지 않습니다.
6. 키 준비: 키는 모듈 로드 시 _load_secret()이 bytes.fromhex 로 32 바이트로 바꿉니다.
   키가 비었거나 예시 값, 형식 오류가 발생하면 서버가 시작되지 않습니다.
7. 출력: **200** {accessToken, expiresIn, user:{id,name,email}}

3.2 브라우저 토큰 저장과 전송

파일 / 함수: frontend/src/api.js session(3 행), authorizationHeaders(8 행), request(25 행), login(55 행)
/ frontend/src/main.jsx logout(101 행)

코드 발췌

```
3   let session = { accessToken: "", user: null, expiresAt: 0 };
4   export const getSession = () => session;
5   export function clearSession() {
6     session = { accessToken: "", user: null, expiresAt: 0 };
7   }
8   function authorizationHeaders() {
9     // FE-02: 토큰이 있을 때만 Bearer 헤더를 만듭니다. "Bearer undefined"를 보내지 않습니다.
10    return session.accessToken
11      ? { Authorization: `Bearer ${session.accessToken}` }
12      : {};
13  }
```

```
41    const data = await readBody(response);
42    // request() 내부
43    if (!response.ok) {
44      // FE-03: 401이면 메모리의 로그인 상태를 해제합니다. 403은 로그인 상태를 유지합니다.
45      if (response.status === 401) clearSession();
46      const error = new Error(
47        data?.message || data?.code || `HTTP ${response.status}`,
48      );
49      error.status = response.status;
50      error.code = data?.code;
```

동작과 응답 코드 설명

1. 저장
   (1) 로그인 성공 응답의 토큰과 user 를 모듈 변수 session 에만 저장합니다.
   (2) localStorage/sessionStorage 는 쓰지 않으므로 새로고침하면 로그인이 해제됩니다.
2. 전송
   (1) 모든 요청은 request()를 거칩니다.
   (2) 토큰이 있을 때만 Authorization: Bearer <JWT>를 붙이므로 Bearer undefined 가 생기지 않습니다.
3. 401 처리
   (1) 401 이 오면 clearSession()으로 토큰과 사용자를 지웁니다.
   (2) 화면은 describe()가 code 별 안내를 보여줍니다.
   (INVALID_CREDENTIALS / TOKEN_REQURIED / INVALID_TOKEN / TOKEN_EXPIRED)
4. 403 처리: 로그인 상태를 유지한 채 권한 부족만 안내합니다.

5. 204 처리: readBody()가 JSON 파싱을 하지 않습니다.
6. 로그아웃: logout()이 clearSession()과 화면 상태를 즉시 초기화합니다.

서버 인증과 작성자 권한

3.3 서버 JWT 검증

파일 / 함수: backend/auth.py _bearer_token(57 행), current_user_id(66 행)

코드 발췌

```python
def current_user_id(request):
    # BE-02: Bearer 추출 → 서명/알고리즘/만료/클레임 검증 → 실제 사용자 확인.
    token = _bearer_token(request)
    try:
        claims = jwt.decode(
            token,
            SECRET,
            algorithms=[ALGORITHM],
            issuer=ISSUER,
            audience=AUDIENCE,
            options={"require": ["exp", "iss", "aud", "sub"]},
        )
    except jwt.ExpiredSignatureError:
        raise ApiError(
            401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요."
        )
    except jwt.InvalidTokenError:
        raise ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.")
    sub = claims["sub"]
    if not isinstance(sub, str) or not re.fullmatch(r"[1-9][0-9]*", sub):
        raise ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.")
    user = find_public(int(sub))
    if not user:
        raise ApiError(401, "INVALID_TOKEN", "사용자를 확인할 수 없습니다.")
    return user["id"]
```

동작과 응답 코드 설명

1. 헤더 확인
   (1) _bearer_token 이 Authorization 헤더를 확인합니다.
   (2) 헤더가 없거나, Bearer 가 아니거나, 값이 비어있으면 **401 TOKEN_REQURIED** 입니다.
2. 라이브러리 검증: jwt.decode 가 다음을 한 번에 검증합니다. payload 읽기만 하는 decode 가 아닙니다.
   (1) 허용 알고리즘 (HS256 고정: alg=none, HS512 거부)
   (2) 서명
   (3) exp 만료
   (4) iss/aud 값
   (5) 필수 클레임 존재
3. 실패 응답: 만료는 **401 TOKEN_EXPIRED**, 나머지 오류는 **401 INVALID_TOKEN** 입니다.
4. 사용자 확인: sub 가 양의 정수 문자열인지, 실제 계정이 있는지 확인합니다.
   둘 중 하나라도 아니면 **401 INVALID_TOKEN**
5. 사용: 검증된 사용자 ID 만 반환해 /me, 등록,수정,삭제에 씁니다.
6. 응답 헤더: 모든 401 에는 app.py 의 handle_error 가 WWW-Authenticate: Bearer 를 붙입니다.

3.4 작성자 권한 검사

파일 / 함수: backend/auth.py assert_owner(93 행)
backend/app.py create(122 행), update(132 행), delete(142 행)

코드 발췌

```python
def assert_owner(user_id, author_id):
    # BE-03: 검증된 로그인 사용자와 DB에 저장된 작성자를 비교한다.
    if user_id != author_id:
        raise ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니다.")
```

```python
@app.put("/api/posts/{id}")
async def update(id: str, request: Request):
    user_id = current_user_id(request)        # 인증 (401)
    pid = post_id(id)                         # ID 형식 (400)
    post = existing(pid)                      # 글 존재 (404)
    assert_owner(user_id, post["authorId"])   # 작성자 권한 (403)
    return store.update_post(pid, await fields(request, dict(title=100, content=5000)))
                                              # 입력 (400) / 변경
```

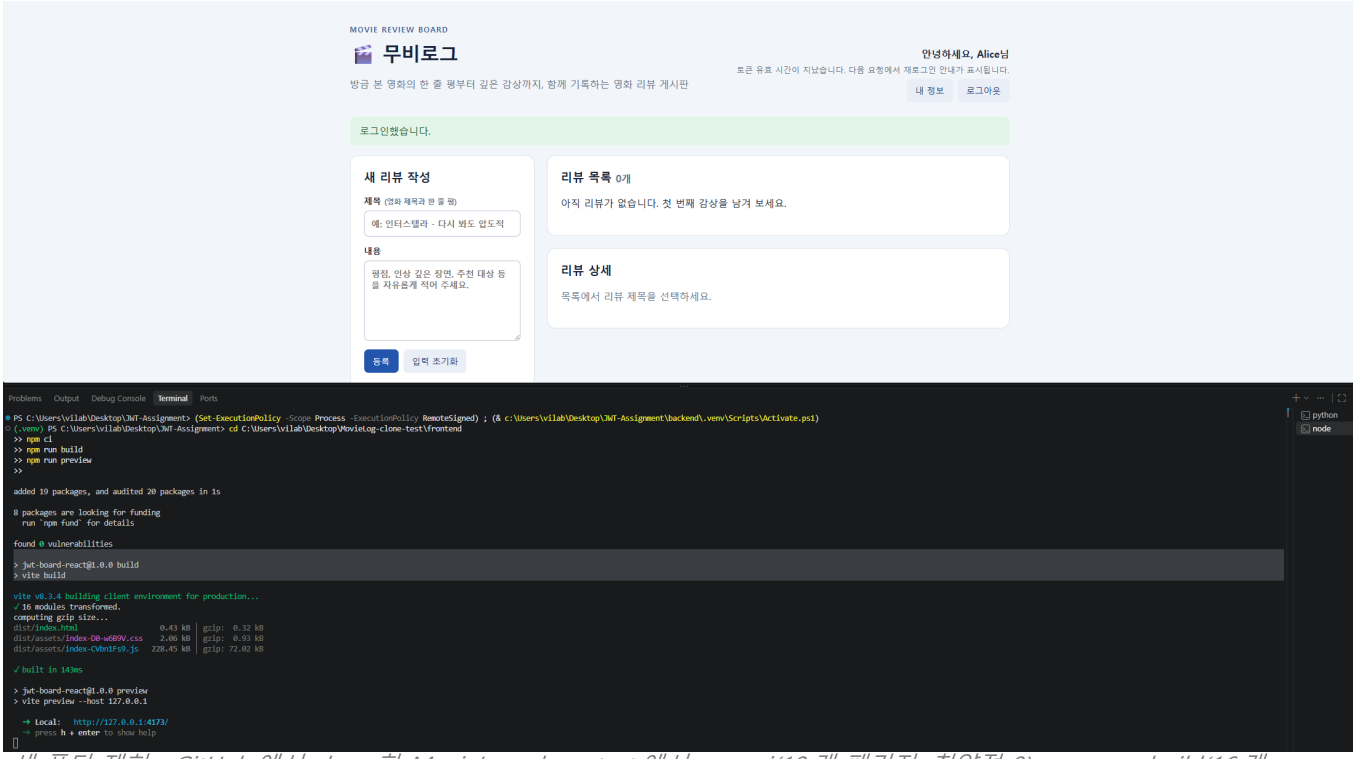
동작과 응답 코드 설명

1. 비교 대상: 토큰에서 검증된 사용자 ID 와 DB 에 저장된 글의 authorId 를 비교합니다.
2. 401 과 403 의 차이
   (1) 401 은 "누구인지 확인할 수 없음" (토큰 없음/무효/만료)입니다.
   (2) 403 은 "누구인지 확인됐지만 이 글을 바꿀 권한이 없음" 입니다. [403 에서 로그아웃하지 않습니다.]
3. 작성자 위조 방지
   (1) 등록 시 create 는 current_user_id 가 돌려준 ID 를 작성자로 저장합니다.
   (2) fields()는 title/content 외의 필드(authorId, userId, id 등)가 오면 **400 BAD_INPUT** 으로 거부합니다.
   (3) 수정 시 update_post 는 title, content, updatedAt 만 바꾸므로 작성자와 createdAt 이 유지됩니다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

1. 실행 환경: [Windows 11 Education / Python 3.12.10, Node.js v24.15.0 / Chrome 154.0.8037.98]
2. 실행 명령
   (1) /backend: ..venv₩Scripts₩python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
   (2) /frontend: npm ci → npm run build → npm run preview
3. 로컬 배포 확인
   (1) 새 폴더에서 설치한 절차: git clone https://github.com/zohijin-k/MovieLog README 절차대로 진행
   (2) backend 에서 가상환경 생성, pip install -r requirements.txt, .env.example 을 .env 로 복사
   (3) 새 JWT_SECRET 설정 후 uvicorn 으로 8000 번 실행
   (4) frontend 에서 npm run preview 로 4173 번 실행했다.



새 폴더 재현 - GitHub 에서 clone 한 MovieLog-clone-test 에서 npm ci(19 개 패키지, 취약점 0), npm run build(16 개 모듈, 종료 성공), npm run preview 실행. 새로 생성된 DB 라 리뷰 목록 0 개이며 Alice 로그인 성공

4. 테스트 일시 및 코드 버전: 2026-10-19 (새 폴더 재현 2026-10-20)
   / main 11a77cb / (주석 정리 후 1af58eb)
5. 토큰 TTL: [일반 테스트 90 초 / 만료 테스트(T10) 3 초]
   (PowerShell 에서 $env:TOKEN_TTL="3" 설정 후 서버 재시작)
   (테스트 후 Remove-Item Env:TOKEN_TTL 로 복구)

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 상태에서 GET /api/posts, GET /api/posts/{id} 요청	200 OK	200	PASS	docs/evidence/Test/T01.png
T02	Alice 가 POST /api/auth/login (정상 자격증명)	200 OK, accessToken/user 반환	200	PASS	docs/evidence/Test/T02.png docs/evidence/Test/T02_screen.png
T03	Alice 가 POST /api/auth/login (틀린 비밀번호) / 없는 이메일로 로그인	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	PASS	docs/evidence/Test/T03.png
T04	비로그인 상태에서 GET /api/me, POST /api/posts 요청	401 TOKEN_REQUIRED, WWW-Authenticate: Bearer	401 / TOKEN_REQUIRED	PASS	docs/evidence/Test/T04.png
T05	Alice 로그인 후 POST /api/posts, GET /api/me	201 Created, authorId=1 / 200 OK, Alice 반환	201 / 200	PASS	docs/evidence/Test/T05.png
T06	Alice 가 본인 게시글 PUT (수정), DELETE (삭제)	200 OK (수정 본문 반환) / 204 No Content	200 / 204	PASS	docs/evidence/Test/T06.png
T07	Bob 이 Alice 게시글 PUT (수정 시도)	403 FORBIDDEN, 게시글 변경 없음	403 / FORBIDDEN	PASS	docs/evidence/Test/T07.png
T08	Bob 이 Alice 게시글 DELETE (삭제 시도)	403 FORBIDDEN, 게시글 존재 유지	403 / FORBIDDEN	PASS	docs/evidence/Test/T08.png
T09	서명 첫 글자를 바꾼 토큰으로 GET /api/me, POST/api/posts, PUT · DELETE/api/posts/11	401 INVALID_TOKEN	4 건 모두 401 / INVALID_TOKEN	PASS	docs/evidence/Test/T09.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T10	TOKEN_TTL=3 으로 재시작, 로그인 5 초 후 GET /api/me, POST/api/posts, PUT・DELETE/api/posts/17	401 TOKEN_EXPIRED, 프론트 자동 로그아웃	4 건 모두 401 / TOKEN_EXPIRED	PASS	docs/evidence/Test/T10.png
T11	Alice 로그인 후 로그아웃, GET /api/me	401 TOKEN_REQUIRED (Authorization 헤더 없음)	401 / TOKEN_REQUIRED	PASS	docs/evidence/Test/results.md docs/evidence/Test/T11.png docs/evidence/Test/browser_T02_T11_T16.txt
T12	Bob 이 POST /api/posts 에 authorId:1, userId:1 을 각각 추가	400 BAD_INPUT, 게시글 미생성	2 건 모두 400 / BAD_INPUT, 위조 글 미생성	PASS	docs/evidence/Test/results.md
T13	Alice 가 빈 제목 / 공백 본문, 제목 101 자, 본문 5001 자로 작성, 수정 시 빈 제목	400 BAD_INPUT	5 건 모두 400 / BAD_INPUT	PASS	docs/evidence/Test/results.md
T14	GET /api/posts/999999, Alice 토큰으로 PUT・DELETE/api/posts/999999, GET/api/posts/abc, /api/posts/0	404 NOT_FOUND / 400 BAD_INPUT	404 / NOT_FOUND 3 건, 400 / BAD_INPUT 2 건	PASS	docs/evidence/Test/results.md
T15	게시글 작성 후 서버 재시작, GET /api/posts/15	재시작 전후 동일 데이터 유지	200, 제목・작성자・updatedAt 동일	PASS	docs/evidence/Test/results.md
T16	로그인 후 브라우저 새로고침 (F5)	게시글 목록 유지, 로그인 상태 초기화	200, 로그인 해제	PASS	docs/evidence/Test/T16_after.png docs/evidence/Test/T16_before.png
T17	비로그인 PUT・DELETE / Bob 이 본인 글 작성,수정,삭제	401 TOKEN_REQUIRED / 201·200·204	401 / TOKEN_REQUIRED, 201 / 200 / 204	PASS	docs/evidence/Test/results.md

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회	200	GET /api/posts , GET /api/posts/4 → 200 / 해당 없음	docs/evidence/Test/T01.png
정상 로그인	200	POST /api/auth/login(Alice) → 200 / 해당 없음, expiresIn 90	docs/evidence/Test/T02.png , /T02_screen.png
정상 수정	200	Alice PUT /api/posts/12 → 200 / 해당 없음 Bob PUT /api/posts/16 → 200 / 해당 없음	docs/evidence/Test/T06.png , /results.md (T17)
새 글 등록	201 및 Location	POST /api/posts → 201 Location: /api/posts/11 (T05) /api/posts/15 (T15), /api/posts/16 (T17)	docs/evidence/Test/T05.png , /results.md (T15, T17)
본인 글 삭제	204 및 빈 본문	Alice DELETE /api/posts/12 → 204, 본문 없음 / Bob DELETE /api/posts/16→ 204, 본문 없음	docs/evidence/Test/T06.png , /results.md (T17)
입력 오류	400 BAD_INPUT	빈 제목, 공백 본문, 제목 101 자, 본문 5001 자, 수정 시 빈 제목 → 5 건 모두 400 / BAD_INPUT	docs/evidence/Test/results.md (T13)
추가 작성자 필드	400 BAD_INPUT	Bob POST /api/posts 에 authorId:1 → 400 / BAD_INPUT userId:1 → 400 / BAD_INPUT	docs/evidence/Test/results.md (T12)
잘못된 계정	401 INVALID_CREDENTIALS	틀린 비밀번호, 없는 이메일 → 둘 다 401 INVALID_CREDENTIALS, 토큰 미발급	docs/evidence/Test/T03.png
토큰 없음	401 TOKEN_REQUIRED	GET /api/me, POST /api/posts, PUT·DELETE /api/posts/11 → 401 / TOKEN_REQUIRED	docs/evidence/Test/T04.png , /results.md (T11, T17)
헤더 형식 오류	401 TOKEN_REQUIRED	Bearer 접두사 없음, Basic 방식 → 401 / TOKEN_REQUIRED	docs/evidence/Test/check_claims_1019.txt

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
변조	**401 INVALID_TOKEN**	서명 첫 글자 변조 후 GET /api/me, POST /api/posts, PUT·DELETE /api/posts/11 → 4 건 모두 401 / INVALID_TOKEN	docs/evidence/Test/T09.png
클레임 오류	**401 INVALID_TOKEN**	HS512, alg=none, 다른 키, iss·aud 불일치, exp·sub·iss·aud 누락, sub 형식 오류, 없는 사용자 → 13 건 모두 401 / INVALID_TOKEN	docs/evidence/Test/check_claims_1019.txt
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	TOKEN_TTL=3, 5 초 후 GET /api/me, POST /api/posts, PUT·DELETE /api/posts/17 → 4 건 모두 401 / TOKEN_EXPIRED	docs/evidence/Test/T10.png , /results.md (T10)
다른 작성자의 변경	**403 FORBIDDEN**	Bob PUT /api/posts/11→ 403 / FORBIDDEN, Bob DELETE /api/posts/11 → 403 / FORBIDDEN	docs/evidence/Test/T07.png , /T08.png , results.md(T07, T08)
없는 글	**404 NOT_FOUND**	GET /api/posts/999999 → 404, Alice PUT·DELETE /api/posts/999999 → 404 / NOT_FOUND, 삭제 후 GET /api/posts/12→ 404	docs/evidence/Test/T06.png , /results.md (T14)

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]
보호 API 가 401 을 반환한 모든 요청에서 응답 헤더에 WWW-Authenticate: Bearer 가 포함된 것을 확인했습니다. 토큰 없이 보낸 GET /api/me 와 POST /api/posts(401 TOKEN_REQUIRED, T04.png), 서명을 변조한 토큰으로 보낸 GET /api/me, POST /api/posts, PUT·DELETE /api/posts/11(401 INVALID_TOKEN, T09.png)이 여기에 해당합니다. 만료 토큰으로 보낸 같은 네 요청(401 TOKEN_EXPIRED, results.md T10 행)과 토큰 없이 보낸 PUT·DELETE /api/posts/11(results.md T17 행)도 마찬가지입니다. 로그인 실패(401 INVALID_CREDENTIALS)에도 같은 헤더가 포함되었습니다(T03.png).

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]
401 이면 프론트의 api.js request()가 clearSession()을 호출해 메모리의 토큰과 사용자 정보를 지웁니다. 그래서 화면 상단이 "로그인하지 않은 상태 · 리뷰 읽기만 가능"으로 바뀌고, JSON code 에 맞는 안내가 표시됩니다. 만료 토큰으로 글을 작성했을 때는 "로그인 시간이 만료되어 로그아웃되었습니다. 다시 로그인하세요."가 표시되었습니다(T10.png). 로그아웃 후 내 정보를 조회했을 때는 "로그인이 필요합니다. 로그인 후 다시 시도하세요."가 표시되었습니다(T11.png).

403 은 인증은 성공했지만 권한이 없는 경우라서 세션을 지우지 않습니다. 따라서 로그인 상태가 유지되고, "권한이 없습니다. 본인이 작성한 리뷰만 수정/삭제할 수 있습니다." 안내만 표시됩니다(main.jsx describe()). 화면에서는 canEdit()가 다른 사람의 글에 수정/삭제 버튼을 표시하지 않습니다. 그래서 403 은 Bob 의 토큰으로 서버에 직접 요청해 확인했습니다(T07.png, T08.png).

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

Bob 이 Alice 의 11 번 글을 수정/삭제하려다 403 FORBIDDEN 을 받은 뒤, 토큰 없이 GET /api/posts/11 로 다시 조회했습니다. 제목 'Alice original', 작성자 authorId 1, updatedAt 2026-10-19T05:51:35.946765+00:00 으로 작성 시점과 같았습니다(results.md T07,T08 행). 변조 토큰 요청(T09), 입력 오류(T13), 만료 토큰 요청(T10) 뒤에 재조회한 결과도 같은 값이었습니다. 만료 토큰으로 수정/삭제를 시도한 17 번 글도 내용과 updatedAt 이 바뀌지 않았습니다. 목록 조회에서도 실패한 요청의 글('bad', 'forged', 'expired')은 생성되지 않았습니다.

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

같은 요청 본문 {"title":"t","content":"c"}로 PUT /api/posts/999999 를 보냈습니다. 토큰 없이 보냈을 때는 401 TOKEN_REQUIRED(WWW-Authenticate: Bearer 포함), Alice 의 정상 토큰으로 보냈을 때는 404 NOT_FOUND 가 반환되었습니다. 이어서 다음 세 경우도 확인했습니다.

- Bob 이 Alice 의 11 번 글에 공백 제목으로 수정 요청: 403 FORBIDDEN
- Alice 가 자기 글에 같은 입력으로 요청: 400 BAD_INPUT
- Bob 의 만료 토큰으로 같은 요청: 401 TOKEN_EXPIRED

명세의 처리 순서인 인증 → 글 존재 확인 → 작성자 권한 → 입력 검증대로 응답한 것을 확인했습니다(results.md 겹침 행).

테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

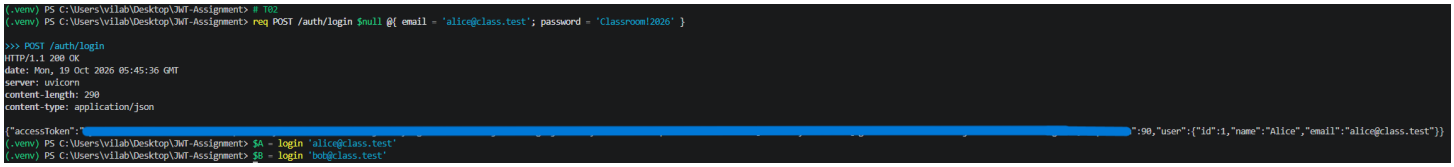


그림 1. T02 – POST /api/auth/login(Alice) → 200 OK. 응답에 accessToken(가림), expiresIn 90, user {id 1, name Alice}가 포함됨

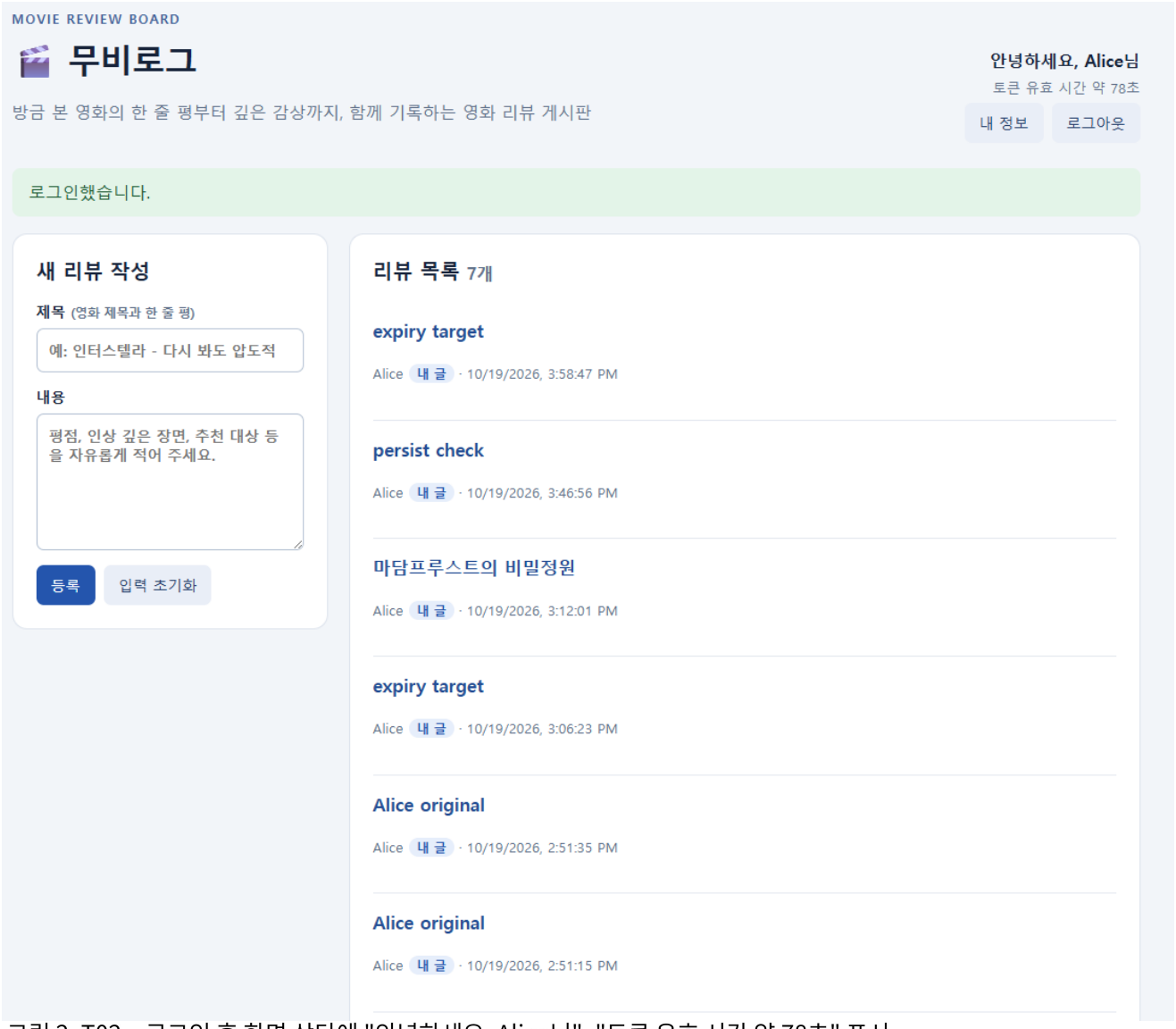


그림 2. T02 – 로그인 후 화면 상단에 "안녕하세요, Alice님", "토큰 유효 시간 약 78초" 표시

그림 3. T05 – Alice 토큰으로 POST /api/posts → 201 Created, Location: /api/posts/11, authorId 1. 이어서 GET /api/me → 200, Alice

브라우저가 보낸 요청 헤더는 browser_T02_T11_T16.txt 에 기록했습니다. 로그인 후 GET /api/me 요청에는 Authorization: Bearer <TOKEN>이 첨부되어 200 이 반환되었습니다. 로그아웃 후 같은 요청에서는 Authorization 헤더가 빠지고 401 TOKEN_REQUIRED 가 반환되었습니다(T11).

다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

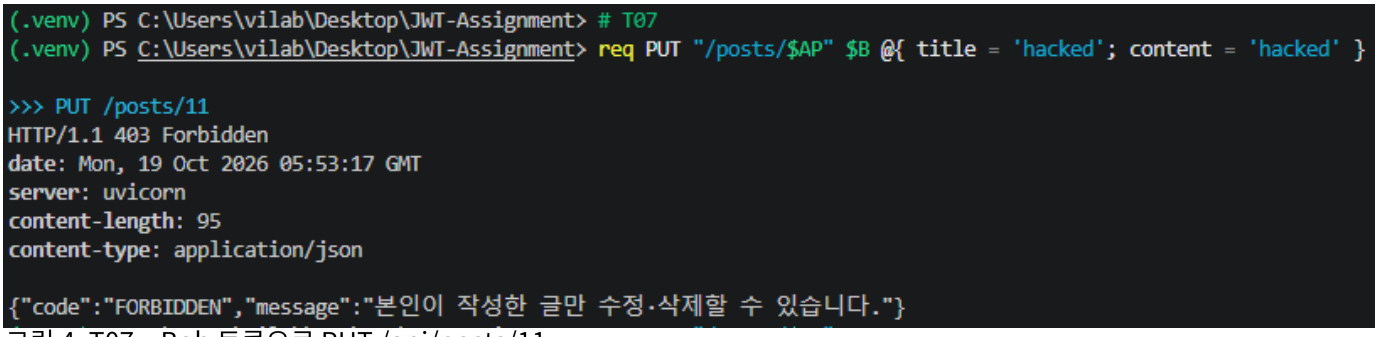


그림 4. T07 – Bob 토큰으로 PUT /api/posts/11

→ 403 Forbidden, code FORBIDDEN, "본인이 작성한 글만 수정·삭제할 수 있습니다."

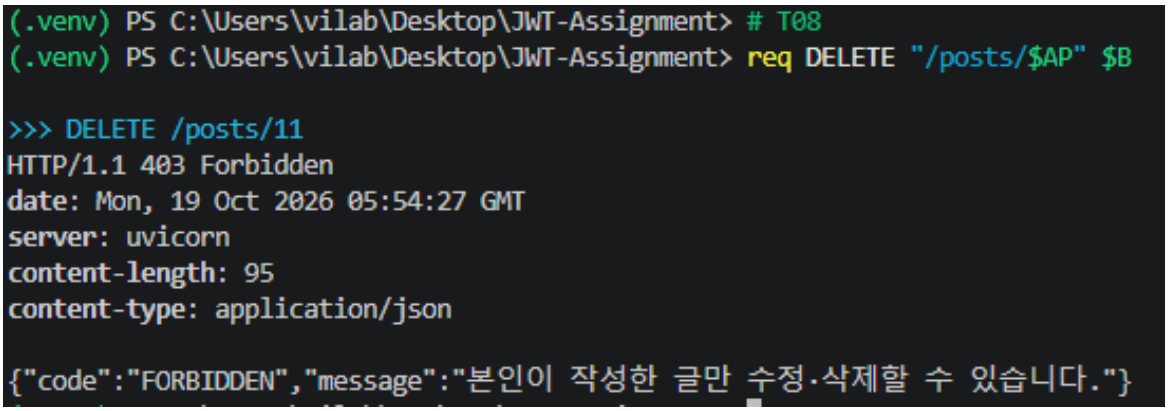


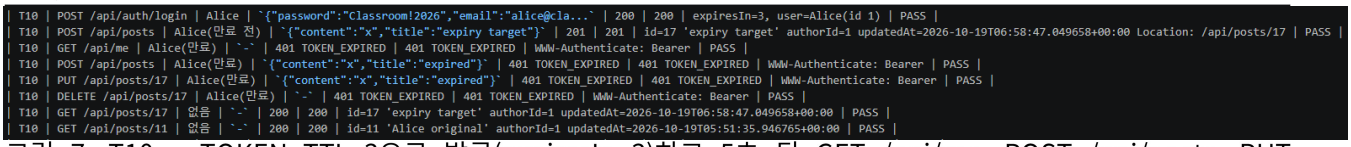
그림 5. T08 – Bob 토큰으로 DELETE /api/posts/11 → 403 Forbidden, code FORBIDDEN

두 요청이 실패한 뒤 GET /api/posts/11 로 다시 조회했습니다. 제목 'Alice original', authorId 1, updatedAt 2026-10-19T05:51:35.946765+00:00 으로 작성 시점과 같아, 원문이 그대로 유지된 것을 확인했습니다(results.md T07·T08 행). 수정·삭제 버튼은 화면에서 숨겨져 있으므로 Bob 의 정상 토큰으로 서버에 직접 요청해 확인했습니다.

토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

그림 6. T09 – 서명 첫 글자를 바꾼 토큰으로 GET /api/me, POST /api/posts, PUT /api/posts/11, DELETE /api/posts/11 → 4건 모두 401 INVALID_TOKEN, WWW-Authenticate: Bearer. 이후 재조회에서 11번 글은 변경 없음, 'bad' 글은 생성되지 않음



T10	POST /api/auth/login	Alice	`{"password":"Classroom!2026","email":"alice@cla...`	200	200 | expiresIn=3, user=Alice(id 1) | PASS |
T10	POST /api/posts	Alice(만료 전)	`{"content":"x","title":"expiry target"}`	201	201 | id=17 'expiry target' authorId=1 updatedAt=2026-10-19T06:58:47.049658+00:00 Location: /api/posts/17 | PASS |
T10	GET /api/me	Alice(만료)	`-`	401 TOKEN_EXPIRED	401 TOKEN_EXPIRED | WWW-Authenticate: Bearer | PASS |
T10	POST /api/posts	Alice(만료)	`{"content":"x","title":"expired"}`	401 TOKEN_EXPIRED	401 TOKEN_EXPIRED | WWW-Authenticate: Bearer | PASS |
T10	PUT /api/posts/17	Alice(만료)	`{"content":"x","title":"expired"}`	401 TOKEN_EXPIRED	401 TOKEN_EXPIRED | WWW-Authenticate: Bearer | PASS |
T10	DELETE /api/posts/17	Alice(만료)	`-`	401 TOKEN_EXPIRED	401 TOKEN_EXPIRED | WWW-Authenticate: Bearer | PASS |
T10	GET /api/posts/17	없음	`-`	200	200 | id=17 'expiry target' authorId=1 updatedAt=2026-10-19T06:58:47.049658+00:00 | PASS |
T10	GET /api/posts/11	없음	`-`	200	200 | id=11 'Alice original' authorId=1 updatedAt=2026-10-19T05:51:35.946765+00:00 | PASS |

그림 7. T10 – TOKEN_TTL=3으로 발급(expiresIn 3)하고 5초 뒤 GET /api/me, POST /api/posts, PUT /api/posts/17, DELETE /api/posts/17 → 4건 모두 401 TOKEN_EXPIRED. 17번 글은 변경 없음

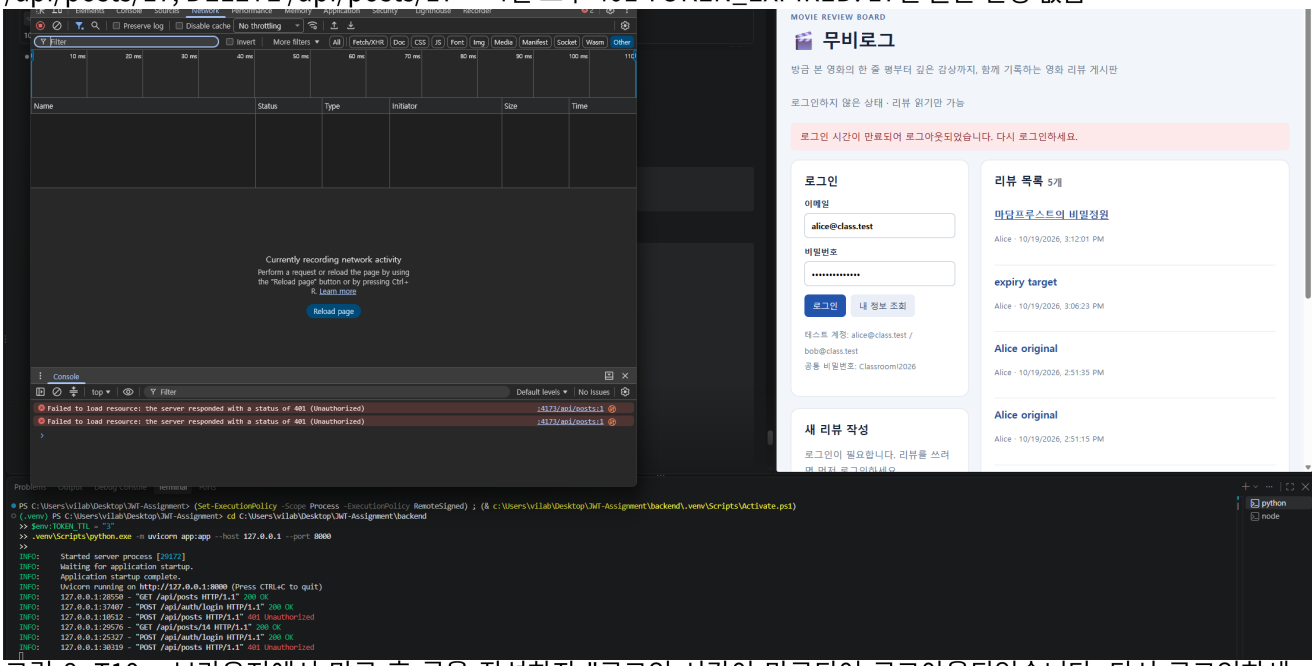


그림 8. T10 – 브라우저에서 만료 후 글을 작성하자 "로그인 시간이 만료되어 로그아웃되었습니다. 다시 로그인하세요." 안내와 함께 로그인하지 않은 상태로 전환. 콘솔과 서버 로그에 POST /api/posts 401 기록

자동 검사와 로컬 배포 증거

[자동 검사 결과를 첨부했다면 실행 명령과 의미를 설명]

실행 명령: backend/.venv/Scripts/python.exe tools/check_claims.py (2026-10-19 16:10, TOKEN_TTL=90 서버)

결과: 18/18 PASS (check_claims_1019.txt)

이 스크립트는 로컬 .env의 실습 키로 서명은 올바르지만 클레임이 잘못된 토큰을 직접 만들어 보냅니다. HS512 알고리즘, alg=none, 다른 키 서명, iss·aud 불일치, exp·sub·iss·aud 누락, sub 형식 오류, 없는 사용자는 모두 401 INVALID_TOKEN이었습니다. 정상 서명의 만료 토큰은 401 TOKEN_EXPIRED, 헤더 없음·Bearer 접두사 없음·Basic 방식은 401 TOKEN_REQUIRED로 거부되었습니다. payload만 바꾸면 서명 검사에서 걸리기 때문에, 이 스크립트로 클레임 검사가 따로 동작하는지 확인했습니다. 이 결과는 T01~T17의 수동 테스트를 대체하지 않는 보조 증거입니다.

[빌드 성공 로그 / 4173번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

빌드 성공 로그: docs/evidence/Test/build_log.txt. npm run build(vite v8.3.4, Node v24.15.0) 결과 16개 모듈 변환, dist/index.html과 JS·CSS 생성, 종료 코드 0

4173번 API 연결: T10.png와 T02_screen.png. npm run preview로 실행한 http://127.0.0.1:4173 화면이 /api 요청을 8000번 백엔드로 전달해 리뷰 목록을 표시함. browser_T02_T11_T16.txt에 4173을 거친 /api/posts, /api/auth/login, /api/me 요청과 응답이 기록됨

재시작 후 데이터 유지: results.md T15 행. 백엔드를 Ctrl+C로 종료하고 다시 실행한 뒤 GET /api/posts/15를 조회했고, 제목 'persist check', authorId 1, updatedAt 2026-10-19T06:46:56.606301+00:00이 재시작 전과 같음

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
claude code (Opus 5.5)	설계 및 생성, 테스트 cli 작성 및 보고서 문장 정리 보조	1. 개발의 전 부분 2. 보고서의 전 항목	그대로 반영 및 수정 후 반영

대표 사례 1: 요구사항 프롬프트와 과제 명세의 충돌 확인
도구 / 날짜: Claude Code (Opus 5.5) / 2026-10-12
요청한 내용: "직접 작성한 상세 프롬프트 + React + FastAPI 전체 구현을 요청했음. 프롬프트에는 python-jose·bcrypt 사용, 테스트 계정 alice@example.com / password123, 사용자 ID "user-alice", iss=board-api·aud=board-client, 128자리 JWT_SECRET이 적혀 있었음."
AI가 제안한 내용: 구현 전에 폴더의 docs/02_API명세.md, docs/03_구현체크리스트.md, tools/check_api.py, 제공 시작 코드를 먼저 읽음. 그 결과 프롬프트와 명세가 5군데 다르다고 알림.
내가 확인하고 수정한 내용:
구현이 끝나고 과제의 명세와 실제 구현이 맞는지 비교하며 검증함
검증 방법과 결과: python tools/check_api.py --mode final → All 40 checks passed. 프롬프트의 계정으로 구현했다면 이 스크립트의 로그인 단계부터 실패했을 것임. [직접 실행한 결과로 확인]

대표 사례 2: 테스트 시나리오 실행 명령 작성과 토큰 만료 오류 해결
도구 / 날짜: Claude Code (Opus 5.5) / 2026-10-19
요청한 내용: "테스트시나리오md를 수행하기 위한 모든 cli를 주세요." 이어서 T05가 실패하자 "05에서 잘못된거같죠?"라고 원인을 물었음.
AI가 제안한 내용: API 명세와 백엔드 설정(.env, 포트 8000)을 읽고, T01~T17을 실행하는 PowerShell 명령을 제안함. PowerShell 5.1에서 curl.exe로 JSON을 보내면 따옴표가 깨지기 때문에, 요청 본문을 임시 파일로 저장해 전송하는 req 함수와 토큰을 받는 login 함수를 함께 제안함.
실제 반영 위치: 저장소 코드에는 반영하지 않음. 테스트용 터미널(PowerShell)에서만 사용함. 보고서 4장 테스트 결과의 수행 절차로 반영.
내가 확인하고 수정한 내용:
T05 글 작성이 201이 아니라 401 TOKEN_EXPIRED로 실패함. 로그인 시각(05:45:36)과 요청 시각(05:48:28)을 비교해, 서버 오류가 아니라 TOKEN_TTL 90초가 지난 것이 원인임을 확인함. 실패한 상태에서 $AP = latestId가 실행돼 테스트 대상 글 ID가 기존 글(4번)로 잘못 잡힌 것도 발견함. 매 테스트 블록 시작 시 토큰을 새로 받는 fresh 함수를 추가하고, T05부터 다시 실행함. T10 서버 재시작 명령이 프로젝트 루트에서는 .venv 경로를 찾지 못해 실패함. backend 폴더로 이동하는 명령을 추가해 수정함.
검증 방법과 결과: T05 재실행 결과 POST /api/posts → 201 Created, Location: /api/posts/11, authorId 1. GET /api/me → 200, Alice(T05.png). 이후 T06~T09도 같은 방식으로 실행해 기대 결과를 확인함(T06.png~T09.png).

대표 사례 3: 테스트 결과를 표로 자동 기록하는 함수
도구 / 날짜: Claude Code (Opus 5.5) / 2026-10-19
요청한 내용: "캡처본 넣는데 불편합니다. 파일로 만드는 방법이 있나요?" – 요청마다 화면을 캡처해 보고서에 넣는 방식이 번거롭다고 요청함.
AI가 제안한 내용: 요청 하나를 보낼 때마다 테스트 ID, 메서드·URL, 사용자, 요청 본문, 예상 결과, 실제 HTTP와 code, Location·WWW-Authenticate 헤더, 응답 데이터 요약, PASS/FAIL을 마크다운 표 한 줄로 기록하는 t 함수를 제안함. 토큰 원문은 표에 남지 않도록 함.
실제 반영 위치: docs/evidence/Test/results.md (T07·T08 재조회, T10~T17, 여러 오류가 겹친 경우). 보고서 4장 HTTP 응답 확인 기록 표.
내가 확인하고 수정한 내용:
처음 실행한 T12 첫 행이 400이 아니라 401 TOKEN_EXPIRED(FAIL)로 기록됨. 테스트 사이에 시간이 지나 Bob 토큰이 만료된 것이 원인이라 토큰을 다시 받아 재실행함. 보고서 표에서는 만료로 생긴 FAIL 행과 중복된 READY 행을 삭제함. 증거 폴더를 Test로 옮긴 뒤 함수가 이전 경로에 새 파일을 만든 것을 발견함. 두 파일을 하나로 합치고 행을 테스트 ID 순서로 정리함.
검증 방법과 결과: 정리된 results.md의 모든 행이 PASS. T15는 서버 재시작 전후 GET /api/posts/15의 updatedAt(2026-10-19T06:46:56.606301+00:00)이 같아 데이터 유지를 확인함.
겹침 조건은 401 TOKEN_REQUIRED → 404 NOT_FOUND → 403 FORBIDDEN → 400 BAD_INPUT 순서로 명세와 일치함.

대표 사례 4: 브라우저 확인 자동화
도구 / 날짜: Claude Code (Opus 5.5) / 2026-10-19
요청한 내용: T02 화면, T11 헤더, T16 새로고침 캡처 방법을 몰라 대신 수행을 요청함.
AI가 제안한 내용: puppeteer-core로 Edge를 자동 조작해 4173 프론트에서 로그인, 로그아웃, 내 정보 조회, 새로고침을 수행하고 화면 캡처와 요청 헤더 로그를 남김(토큰은 <TOKEN>으로 가림).
실제 반영 위치: docs/evidence/Test/T11.png, T16_before.png, T16_after.png, browser_T02_T11_T16.txt. 스크립트는 저장소에 넣지 않음.
내가 확인하고 수정한 내용: 처음 로그에는 로그아웃 후 요청만 있어 헤더가 빠졌다는 대비가 드러나지 않아, 로그아웃 전 내 정보 조회 단계를 추가해 재실행함.
검증 방법과 결과: 로그아웃 전 GET /api/me는 Authorization: Bearer 첨부 → 200, 로그아웃 후에는 헤더 없음 → 401 TOKEN_REQUIRED. 새로고침 전후 리뷰 7개 유지, 로그인만 해제.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
과제 제공 문서 (02_API 명세.md, 05_테스트시나리오.md)	전체 구현, 4 장 테스트	응답 코드/처리순서/테스트 절차
PyJWT Usage, https://pyjwt.readthedocs.io/en/stable/usage.html	backend/auth.py	jwt.encode/decode, algorithms·issuer·audience·require 옵션
FastAPI 문서 https://fastapi.tiangolo.com/	backend/app.py	예외 처리기, Request 사용

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

서버가 요청 본문의 authorId 를 그대로 저장하면, Bob 이 자기 토큰으로 로그인한 채 {"authorId":1}을 보내 Alice 이름으로 글을 쓸 수 있습니다. 같은 방식으로 다른 사람의 글을 자기 글로 바꿀 수도 있습니다.
클라이언트가 보낸 값은 누구나 조작할 수 있으므로 작성자 정보의 근거가 될 수 없습니다.
제 코드는 이 문제를 두 단계로 막습니다.

[1] **추가 필드 거부**: app.py 의 fields()는 허용된 필드(title, content) 외의 키가 하나라도 있으면 400 BAD_INPUT 으로 요청을 거부합니다. 판단은 set(body)-set(limits) 검사로 합니다.
[2] **작성자는 토큰에서만 결정**: create()는 current_user_id(request)가 JWT 검증으로 얻은 사용자 ID 만 store.create_post(data, user_id)에 넘깁니다. update_post()의 UPDATE 문은 title, content, updated_at 만 바꾸고 author_id 는 건드리지 않습니다.

T12 에서 Bob 토큰으로 authorId:1 과 userId:1 을 각각 보냈을 때, 둘 다 400 BAD_INPUT 이었고, 위조 글은 생성되지 않았습니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼 숨김은 화면 표시일 뿐입니다. 공격자는 curl, API 클라이언트, 개발자 도구로 PUT/DELETE 요청을 서버에 직접 보낼 수 있습니다. 프론트의 canEdit()는 사용자 편의를 위한 표시 조건이고 보안 장치가 아닙니다.
실제 권한 검사는 서버 auth.py 의 assert_owner(user_id, author_id)가 합니다. app.py 의 update()와 delete()는 다음 순서로 처리합니다.

(1) current_user_id()로 토큰을 검증해 사용자 ID 를 얻습니다.
(2) existing()으로 DB 에서 글을 조회합니다.
(3) assert_owner()로 검증된 사용자 ID 와 DB 에 저장된 authorId 를 비교합니다. 다르면 403 FORBIDDEN 을 반환합니다.

T07/T08 에서 버튼이 없는 상태로 Bob 의 토큰을 써서 Alice 글에 직접 PUT/DELETE 를 보냈고, 둘 다 403 FORBIDDEN 이 반환되었습니다. 재조회에서 글은 바뀌지 않았습니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

[1] **decode**: 토큰의 header 와 payload 는 Base64URL 로 인코딩만 되어 있어서 키 없이도 누구나 읽을 수 있습니다. decode 만 하면 내용을 읽을 뿐, 누가 만들었는지와 변조 여부는 알 수 없습니다.
[2] **verify**: 서버의 비밀 키로 서명을 다시 계산해 일치하는지 확인하고, 만료와 발급자 같은 클레임까지 검사해야 토큰을 믿을 수 있습니다.

제 코드의 auth.py current_user_id()는 PyJWT 의 jwt.decode()를 키와 검증 옵션과 함께 호출하므로, 이름은 decode 지만 실제로는 다음을 모두 검증합니다.
[1] **알고리즘**: algorithms=["HS256"]으로 고정합니다. HS512 나 alg=none 은 거부됩니다.
[2] **서명**: .env 의 32 바이트 키(SECRET)로 서명을 확인합니다.
[3] **만료**: exp 가 지났으면 ExpiredSignatureError → 401 TOKEN_EXPIRED 를 반환합니다.
[4] **발급자와 대상**: issuer="webservice-class", audience="board-api"와 일치해야 합니다.
[5] **필수 클레임**: options={"require":["exp","iss","aud","sub"]}로 하나라도 없으면 거부합니다.
[6] **sub 형식과 사용자 존재**: 검증 후 sub 가 양의 정수 문자열인지 정규식으로 확인하고, find_public()으로 실제 계정이 있는지 확인합니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[1] **로그아웃**: 프론트 api.js 의 clearSession()이 메모리 변수 session 의 토큰과 사용자 정보를 지웁니다. 이후 요청에는 Authorization 헤더가 붙지 않습니다(T11: 로그아웃 후 GET /api/me → 401 TOKEN_REQUIRED). 서버에는 아무것도 바뀌지 않습니다.
[2] **토큰 만료**: 서버가 exp 를 검사해 401 TOKEN_EXPIRED 를 반환합니다. 프론트는 401 을 받으면 clearSession()을 호출해 로그아웃 상태로 바꾸고 재로그인을 안내합니다(T10).
[3] **새로고침**: 토큰을 localStorage 나 sessionStorage 가 아닌 JavaScript 메모리에만 저장하므로 페이지를 다시 불러오면 사라집니다. 공개 글 목록은 서버 DB 에서 다시 불러오므로 유지됩니다(T16: 새로고침 전후 리뷰 7 개 유지, 로그인만 해제).

이미 복사된 토큰은 로그아웃이나 새로고침과 관계없이 exp 시각까지 유효합니다. 이 서버는 토큰 상태를 저장하지 않는(stateless) 방식이라 발급한 토큰을 개별적으로 취소할 수 없기 때문입니다. 로그아웃은 브라우저의 사본만 지울 뿐입니다. 그래서 TOKEN_TTL 을 90 초로 짧게 두어 유출되더라도 사용할 수 있는 시간을 줄였습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

테스트 중 T05 에서 Alice 토큰으로 글을 작성했는데, 기대한 201 이 아니라 401 TOKEN_EXPIRED 가 나왔습니다. 처음에는 서버의 토큰 검증 코드에 문제가 있다고 생각했습니다.
원인을 확인하려고 로그인 응답의 시각(05:45:36)과 글 작성 요청 시각(05:48:28)을 비교했습니다. 약 2 분 50 초가 지나 TOKEN_TTL 90 초를 넘긴 상태였습니다. 즉 서버는 명세대로 정확히 동작했고, 문제는 토큰을 한 번만 받아 여러 테스트에 계속 쓴 제 방식에 문제가 있었습니다.
이 실패로 인해 이후 테스트 블록마다 토큰을 새로 발급받아 진행했습니다. 재 검증 결과 POST /api/posts → 201, Location: /api/posts/11, GET /api/me → 200 Alice 로 기대 결과와 일치했습니다.
이 경험으로 짧은 TTL 이 보안에는 유리하지만 사용 중 만료가 자주 일어난다는 점을 확인했습니다. 프론트가 401 을 받으면 로그인 상태를 해제하고 안내하도록 구현한 것이 왜 필요한지도 이해했습니다.

현재 남은 문제와 다음에 개선할 점

(1) 서버가 토큰 상태를 저장하지 않아, 로그아웃해도 이미 복사된 토큰은 만료 전까지 사용할 수 있습니다. 토큰 ID 기반 차단 목록이나 짧은 access token 과 refresh token 조합을 도입할 수 있습니다.
(2) TTL 이 90 초로 짧고 갱신 기능이 없어, 긴 글을 쓰는 중에 만료되면 다시 로그인해야 합니다. 만료 직전 알림이나 작성 중인 입력 보존을 개선할 수 있습니다.
(3) 다른 사람 글의 수정 및 삭제 버튼을 숨기기 때문에, 403 응답 화면 처리는 코드로만 확인했고 실제 화면에서는 확인하지 못했습니다.

제출 확인: [✓] 실제 코드와 설명 일치 [✓] 테스트 직접 수행 [✓] AI 활용 및 출처 기록 [✓] 비밀 키와 토큰 제거
GitHub 확인: [✓] 루트 README 작성 [✓] 보고서 PDF 와 증거 업로드 [] README 문서 링크 확인 [✓] 교수자 접근 가능 [✓] 저장소 URL 과 커밋 SHA 제출 []